

sprite component, which you can use to create games.

- Media components include a sound component (which also handles making the phone vibrate), a video player, and camera controls.
- The *ActivityStarter* component lets you fire off an activity, such as another App Inventor application, the camera application, a Web browser/ Web search, or the maps application on the phone.
- The *SpeechRecognizer* and *TextToSpeech* components' purpose is obvious from their names. However, information on the Net says that to achieve effective text-to-speech (and presumably, voice recognition), you need to be running at least Android 1.6, Eclair.

Designing

Figure 3 shows the 'viewer' area where you drop components from the palette in order to use them. As part of the MoleMash tutorial, I have added a *Canvas* component containing an *ImageSprite*, and below the *Canvas* is a *HorizontalArrangement*, containing *Labels* for *Score* and *Misses*. Below these is a *Reset Button*. Below the white canvas area are two non-visible components. (Components can be visible or non-visible. For example, a *Button* component is visible on-screen to the user, who can interact with it. On the other hand, a *Clock* component is non-visible; it is a source of timer events for the application, and not an item that the user interacts with.)

Figure 4 shows both the components tree and the properties pane. The indentation of components below each other in the tree shows a parent (container)/child relationship. For example, the *Mole (ImageSprite)* component is contained in the *MyCanvas* component. The *Rename* and *Delete* buttons have the obvious functions when a component other than *Screen1* is selected. The screen, which is added by default to every new project, cannot be renamed or deleted; you can, however, change the Title that is displayed to the user, as you can see in the properties pane in this image. Also, it is not possible, at present, to have multiple screens in one project—so you can't create multi-screen applications with Inventor. However, there is a workaround: have one AIA app launch another AIA app, as documented at <http://goo.gl/Mh32>.

The Blocks Editor

The Blocks Editor uses the Open Blocks Java library, distributed by MIT's Scheller Teacher Education Program. Open Blocks visual programming is closely related to the Scratch programming language, a project of the MIT Media

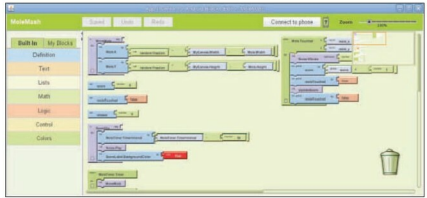


Figure 5: Blocks Editor with MoleMash tutorial open

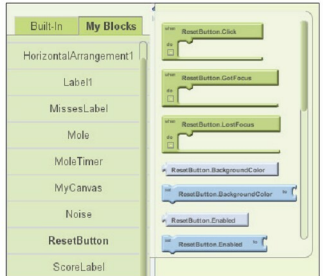


Figure 6: My Blocks tab active, ResetButton drawer open

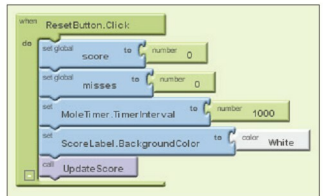


Figure 7: Filled-in Click event handler for ResetButton

Laboratory's Lifelong Kindergarten Group. (Incidentally, if you've seen or played with the Scratch-like Sugar-TurtleArt package [<http://goo.gl/EVZh>], which is available for Ubuntu, and is one of the activities on the OLPC XO, you'll find App Inventor somewhat similar.)

The compiler for the visual blocks 'language' uses the Kawa Language Framework; Kawa's dialect of